

APELLIDOS:

NOMBRE:

PARTE TEÓRICA

Se compone de dos partes: un primer ejercicio de tipo test con 10 preguntas y un segundo con una pregunta de respuesta corta.

Ejercicio 1 (2 Puntos)

Las preguntas acertadas sumarán 1 punto. Las falladas restarán 0.25. Las preguntas no contestadas ni suman ni restan. Hay una única respuesta correcta por pregunta.

- ¿Cuál de las siguientes afirmaciones sobre redes neuronales recurrentes es cierta?
 - No existen bucles de retroalimentación en las neuronas.
 - Los estados ocultos actúan como la memoria de la red.
 - No existe el problema del desvanecimiento del gradiente.
 - Ninguna de las otras respuestas es correcta.
- Si se utiliza un reconocedor de Entidades Nombradas que hace uso de la anotación IOB, y es capaz de reconocer 4 categorías: PER, LOC, ORG y MISC. Al reconocer y clasificar las EN de la frase “León XIV, un Papa de EEUU elegido en Roma para todos.”, ¿cuál sería la salida más correcta (no fijarse en el formato sino en el contenido)?
 - [('León', 'B-PER'), ('XIV', 'B-PER'), ('', 'O'), ('un', 'O'), ('Papa', 'B-PER'), ('de', 'O'), ('EEUU', 'B-LOC'), ('elegido', 'O'), ('en', 'O'), ('Roma', 'B-LOC'), ('para', 'O'), ('todos', 'O')]
 - [('León', 'B-PER'), ('XIV', 'B-PER'), ('', 'O'), ('un', 'O'), ('Papa', 'B-PER'), ('de', 'O'), ('EEUU', 'B-LOC'), ('elegido', 'O'), ('en', 'O'), ('Roma', 'B-LOC'), ('para', 'O'), ('todos', 'O'), ('.', 'O')]
 - [('León', 'B-PER'), ('XIV', 'B-MISC'), ('', 'O'), ('un', 'O'), ('Papa', 'B-PER'), ('de', 'O'), ('EEUU', 'B-LOC'), ('elegido', 'O'), ('en', 'O'), ('Roma', 'B-LOC'), ('para', 'O'), ('todos', 'O'), ('.', 'O')]
 - [('León', 'B-PER'), ('XIV', 'I-PER'), ('', 'O'), ('un', 'O'), ('Papa', 'B-PER'), ('de', 'O'), ('EEUU', 'B-LOC'), ('elegido', 'O'), ('en', 'O'), ('Roma', 'B-LOC'), ('para', 'O'), ('todos', 'O'), ('.', 'O')]
- ¿Cuál de las siguientes afirmaciones sobre transformers es cierta?
 - El decoder es la parte del transformer responsable de generar texto secuencialmente, palabra por palabra.
 - La salida de un encoder es un único vector por cada token de entrada.
 - La salida de un encoder es la entrada para otro encoder o para los decoder.
 - Todas las otras afirmaciones son ciertas.
- En el caso del transformer BERT, cuando se le da una frase como entrada, al tokenizar
 - Solo es capaz de representar con los tokens que estén en el vocabulario
 - Si un token no está en el vocabulario mediante un algoritmo lo divide, a su vez, en subtokens que sí pertenezcan al vocabulario.
 - Si un token no está en el vocabulario añade uno de los tokens especiales que tiene ([CLS] o [SEP])
 - Ninguna de las otras respuestas es correcta.

APELLIDOS:

NOMBRE:

5. ¿Qué beneficios tienen los embeddings densos frente a representaciones one-hot?
 - a. Requieren más espacio de memoria
 - b. Capturan similitudes semánticas entre palabras
 - c. Son más interpretables
 - d. Aumento el tiempo de entrenamiento sin mejorar los resultados
6. ¿Cuál es la principal diferencia entre embeddings estáticos y embeddings contextualizados?
 - a. Los embeddings estáticos son más precisos
 - b. Los embeddings contextualizados asignan diferentes vectores a la misma palabra según el contexto
 - c. Los embeddings estáticos usan transformers bidireccionales
 - d. No hay diferencias entre ellos, solo la implementación
7. ¿Qué riesgo puede implicar un aumento excesivo de datos en PLN?
 - a. Reducción de la capacidad de aprendizaje
 - b. Mejora sistemática del rendimiento
 - c. Incremento de la semántica léxica
 - d. Sobreajuste al texto aumentado y pérdida de generalización
 - e.
8. ¿Qué operación se suele aplicar sobre los embeddings de palabras para generar un *sentence embedding* básico?
 - a. Concatenación de todas las palabras
 - b. Multiplicación cruzada de los vectores
 - c. Suma o media de los vectores de las palabras
 - d. Ninguna de las otras respuestas es cierta
9. ¿Cuál es una diferencia clave entre los Small Language Models (SLMs) y los Large Language Models (LLMs)?
 - a. Los SLMs siempre requieren entrenamiento supervisado, mientras que los LLMs no
 - b. Los SLMs tienen mejor rendimiento en tareas generales que los LLMs
 - c. Los SLMs son más eficientes en consumo de recursos y pueden adaptarse mejor a tareas específicas con pocos datos
 - d. Los SLMs utilizan transformers unidireccionales y los LLMs, redes convolucionales
10. ¿Por qué se utilizan técnicas de padding y truncation en modelos Transformer para PLN?
 - a. Para asegurar que todas las secuencias de entrada tengan la misma longitud
 - b. Para aumentar artificialmente el tamaño del vocabulario
 - c. Para reducir la dimensionalidad de los embeddings
 - d. Ninguna de las otras respuestas es correcta.

APELLIDOS:

NOMBRE:

Ejercicio 2 (1 Punto)

Cuando se utilizan Transformers, ¿Qué opción crees que es mejor, entrenar un transformer desde cero o hacer fine-tuning? Razona tu respuesta e indica cuándo puede ser recomendable utilizar cada opción.

Posible respuesta:

Si se tiene un conjunto de datos muy muy grande y no anotado, es mejor entrenarlo desde cero, porque aprende las relaciones del lenguaje y si los datos son muy variados, puede luego funcionar razonablemente bien para la tarea que sea. Sin embargo, si el conjunto de datos que se tiene para la tarea no es muy grande y está anotado, entonces es mejor hacer fine-tuning y así ya se puede aprovechar lo que el modelo tenía aprendido. Se añadirían las capas que se considere para poder obtener la salida según la tarea correspondiente.

PARTE PRÁCTICA**Ejercicio 3 (2.5 Puntos)**

Dada la frase original 'the dog ran fast', se ha obtenido con un modelo de Word2Vec el siguiente diccionario de tokens más similares a las palabras presentes en dicha frase, donde cada token se acompaña de la similitud con el token que es la key en el diccionario:

```
synonyms = {  
    'the': [('this', 0.59)],  
    'dog': [('dogs', 0.87), ('puppy', 0.81), ('pit bull', 0.78)],  
    'ran': [('went', 0.67), ('raced', 0.65), ('sprinted', 0.63)],  
    'fast': [('quick', 0.57), ('rapidly', 0.55)]  
}
```

- a) [1.25 Puntos] Crea un algoritmo que utilice el diccionario anterior de tokens para aumentar la frase original con perturbaciones a nivel de palabra. Guarda en una lista de Python las frases aumentadas con dicho algoritmo.

Solución:

Hay muchas formas de realizar este ejercicio. Lo importante es que lo propuesto tenga más o menos sentido, y que se esté realizando un reemplazamiento de sinónimos de alguna manera.

Dentro de todas las opciones que hay, una posible puede ser que no se haga aumento de datos de stopwords y que luego al hacer aumento, se coja por ejemplo el primer sinónimo que supere

APELLIDOS:

NOMBRE:

cierto umbral de similitud (que para este ejercicio podría ser > 0.55). El primer sinónimo que lo cumpla dado que en este diccionario están ordenados de mayor a menos similitud. Un ejemplo de este código se puede ver a continuación:

```
from nltk.corpus import stopwords

total_augmented = {sentence}
tokens = sentence.split()

for i in range(len(tokens)):
    word = tokens[i]
    ...if word not in stopwords.words("english"):
        syn, sim = synonyms[word][0]
        if sim > 0.55:
            augmented_i = tokens[:i] + [syn] + tokens[i+1:]
            total_augmented.add(" ".join(augmented_i))

print(len(total_augmented))
total_augmented
```

✓ 0.0s Python

4

```
{'the dog ran fast',
 'the dog ran quick',
 'the dog went fast',
 'the dogs ran fast'}
```

- b) [1.25 Puntos] Tras obtener la lista de frases aumentadas calcula, utilizando el modelo de SBERT 'paraphrase-MiniLM-L6-v2', el grado de similitud semántica entre la frase original y cada una de las frases aumentadas. ¿Se mantiene el significado semántico después de hacer el aumento de datos con respecto a la frase original?

Solución:

Para resolver este apartado hay que hacer varios pasos:

- 1) Se debe de calcular con SBERT el embedding de la frase original y el embedding para cada una de las frases aumentadas.
- 2) Para obtener la similitud semántica entre la frase original y las aumentadas se puede calcular la similitud coseno de los embeddings de la frase original y las frases aumentadas.
- 3) Para responder a la pregunta hay que analizar los valores de similitud coseno obtenidos, si son altos eso significará, por lo general, que se está manteniendo la semántica original tras el aumento de datos.

Un ejemplo de código es el siguiente:

```
from sentence_transformers import SentenceTransformer
```

APELLIDOS:

NOMBRE:

```
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
sentence_emb = model.encode(sentence)

augmented_random_embs= model.encode(augmented_random)

scores_augmented_random = cosine_similarity([sentence_emb],
                                             augmented_random_embs)[0].tolist()
```

Ejercicio 4 (2.5 Puntos)

Se proporciona un código completo que entrena un modelo encoder de tipo Transformer utilizando la biblioteca Keras de Tensorflow. Lo que hace es, utilizando un conjunto muy básico de frases en inglés, clasificarlas según su sentimiento positivo o negativo, indicando una probabilidad también.

Se pide lo siguiente:

- a) **[1.25 Puntos]** ¿Crees que hay algún error en el código? En caso afirmativo indica cuál y cómo hay que corregirlo. Ejecuta el código para ver cómo clasifica cada oración y con qué probabilidad. Anota para cada frase qué sentimiento le asigna y con qué probabilidad.

Solución:

Aunque da un error en tiempo de ejecución, dejar la SimpleRNN cambiando algún parámetro para solucionar el error no es la solución correcta, porque no tiene sentido tener la RNN y no tener una capa de multiatención en la arquitectura transformer. Por lo tanto, habría que sustituir esa capa por lo siguiente:

```
x = MultiHeadAttention(num_heads=2, key_dim=32)(embeddings_layer, embeddings_layer,
embeddings_layer)
```

- b) **[1.25 Puntos]** En el código proporcionado en la primera capa utiliza unos embeddings preentrenados de un modelo Word2Vec. Modifica el código para que en esa primera capa parta de unos embeddings aleatorios. ¿Varían los resultados? ¿Para mejor o para peor?

Solución:

En lugar de coger los embeddings de la matriz de word2vec, para obtener los embeddings aleatorios serviría con lo siguiente:

```
token_embeddings = tf.keras.layers.Embedding(vocab_size, 32)(inputs)
```

APELLIDOS:

NOMBRE:

EVALUACIÓN DE LA PRÁCTICA OBLIGATORIA

[2 puntos]

1. Describe las características del conjunto de datos que se ha utilizado para la práctica:
 - a. ¿Sabrías decir, aproximadamente, cuántos documentos tenía el conjunto de datos proporcionado?
 - b. De las clases en las que estaban categorizados los documentos, ¿cuál es más numerosa?
2. Explica brevemente los enfoques que has seguido en la práctica y cuál es que mejor ha funcionado.
3. ¿Has utilizado aumento de datos? En caso afirmativo, describe cómo y motívalo. En el caso de que no lo hayas utilizado, justifica por qué crees que no sería necesario para esta práctica.
4. ¿Has utilizado Transformers? En caso afirmativo, indica cómo. Y si no lo has utilizado explica las razones de no hacerlo.